目录

OpenSAI 系统架构说明 **1**
1. 总体架构 1
2. 核心组件 2
3. 运行时数据流（一个 DAG 从提交到出结果） 2
4. 智能任务放置（决定 WHERE） 3
5. 按任务资源分配（决定 HOW MUCH） 3
6. 设备执行端：进程池 + 资源手环（跨平台） 3
7. 设备间数据面（QUIC 直传） 4
8. 实时监测与聚合视图 4
9. 任务模型与协同感知能力 4
10. 跨平台与工程质量 5
11. 实测效果（相对随机放置基线） 5
12. 主要可配置项 5

OpenSAI 系统架构说明

OpenSAI 是一个面向边缘协同计算的**分布式 DAG 计算卸载平台**。它把一个应用拆分为带依赖关系的子任务图（DAG），由中心协调器智能地把子任务**放置**到合适的边缘设备执行、按依赖顺序汇总结果；在执行过程中对每个任务进行**算力 / 内存 / 带宽的资源分配与控制**；设备之间通过专用数据面**直接传输**中间结果，支撑协同感知、协同推理等数据密集型场景。

---

1. 总体架构

平台分为三层，各司其职、协同闭环：

层	职责	关键组件
接入层	DAG 任务定义与提交、结果回传、HTTP/SSE 接口	`dag_offload/user_client.py`、`scripts/inject_*.py`
智能调度层	DAG 依赖调度、智能任务放置、按任务资源分配、集群聚合监测	`dag_offload/server.py`、`offloading_brain/offloading_brain.py`
设备执行与网络层	设备执行（进程池 + 绑核 + 内存预算）、设备间数据面、实时资源与链路监测	`dag_offload/device_client.py`、`dag_offload/task_pool.py`、`network/dataplane/`

接入层（提交 DAG、查询结果）
↓ TCP 控制面
智能调度层 • 协调器（DAG 解析 → 依赖调度 → 智能放置 + 资源分配 → 集群聚合监测）
↓ TCP 派发子任务
设备执行与网络层（进程池绑核 / 限内存执行；设备之间经 QUIC 数据面直传中间结果）

控制面走 TCP（设备/客户端与协调器之间）；设备之间的大块中间数据走专用 **QUIC 数据面**直传，二者解耦。

---

2. 核心组件

组件	文件	职责
协调中枢	`dag_offload/server.py`	TCP 服务端、DAG 依赖调度、放置与资源决策、结果汇总、聚合监测视图发布
放置与资源决策（大脑）	`offloading_brain/offloading_brain.py`	成本式智能放置（plan / smart），按真实数据量、链路、负载选设备
设备执行端	`dag_offload/device_client.py`、`run_device.py`	注册与心跳、接收子任务、进程池执行、回传结果与运行指标
进程池与资源控制	`dag_offload/task_pool.py`	worker 进程池、`cpu_affinity` 绑核、内存硬上限、BLAS 线程对齐
算法执行器与算法库	`device/interfaces/actuator.py`、`device/implements/Algorithm_*.py`	动态发现并加载算法、即插即用
设备间数据面	`network/dataplane/quic_server.py`、`quic_client.py`	QUIC 批量传输、令牌桶限速、出口公平平分、链路遥测
容器化执行后端	`device/interfaces/resource_controller.py`	可选的容器化执行（限 CPU/内存、容器复用/回收）
状态存储	`predis/`（Redis）	在线设备表、集群聚合监测视图
API 桥接	`dag_offload/user_client.py`	Flask + SSE 接口（设备/结果/状态查询）
任务模型	`task/bean/*`、`result/bean/*`、`config/*`	`SubTaskInfo`、状态机、结果与配置

---

3. 运行时数据流（一个 DAG 从提交到出结果）

1. **提交**：客户端提交 `dagData`，协调器解析为 `SubTaskInfo` 列表，按依赖建图；前驱全部完成的子任务标记为就绪。
2. **放置**：协调器对就绪子任务做智能放置决策（§4），选定目标设备。
3. **资源分配**：协调器为该任务计算 CPU 核数 / 内存预算 / 传输带宽（§5），随派发报文一并下发。
4. **派发与执行**：设备执行端收到子任务，在 worker 进程池中按下发的核数绑核、按内存预算设上限后执行算法（§6）。
5. **设备间数据传输**：当下游任务的前驱产物在另一台设备时，下游设备经 QUIC 数据面**直接拉取**该中间结果（§7），无需经协调器中转大数据。
6. **回传与推进**：设备回传结果与运行指标；协调器标记完成、解除后继依赖、触发后继子任务。
7. **汇总**：整图完成后，协调器汇总结果回传客户端。

设备执行端另有一条心跳线：周期性上报 CPU/内存/核数/负载等，协调器据此维护在线设备表与监测视图，设备断开即从在线表移除。

---

4. 智能任务放置（决定 WHERE）

“每个就绪子任务放到哪台设备”由协调器决策，策略实现于 `offloading_brain.py`，经环境变量 `OPENSAI_PLACEMENT` 选择：

模式	说明
plan（默认）	前瞻整图规划：DAG 提交时一次性规划全图
smart	自适应贪心：子任务就绪时逐个在线决策
random	随机选在线设备（用于对照评测）

节点可带 `directAllocationId` 指定设备（手动放置）；规划所选设备若掉线，自动回退贪心，保证任务仍能落地。

统一成本模型：把“把节点放到设备 d”的代价折算为预计延迟秒数，选代价最小者：

`cost(d) = W_XFER × Σ 传输秒数(产物送往非本机的相关节点) + W_LOAD × 设备并发负载`
`传输秒数 = 链路时延 + 数据字节数 / 链路带宽`

- **按数据量自适应**：数据大 → 传输项主导 → **就近/聚合**省搬运；数据小 → 负载项主导 → **铺开**抢并行；拐点随真实数据量自动移动。
- **输入全为实测**：链路带宽/时延来自数据面遥测（§7）；设备并发负载来自**预留账本**（已放置未完成的任务数，放置即 +1、完成即 -1，实时防超卖）。
- **plan 的前瞻**：按倒拓扑序（消费者先放）规划，让每个节点偏向其后继所在设备——“产出在将被消费处生产”，连源节点也被拉向汇聚点；依赖链同机放置零并发惩罚，只有相互独立的兄弟节点才竞争负载。

---

5. 按任务资源分配（决定 HOW MUCH）

协调器为每个子任务计算一份资源档位 `{cores, mem_budget_mb, bw_mbps}` 注入派发报文，决策均**按设备上报的“可用”收口**避免超额，由设备端机制执行。

- **算力（CPU 核）**：按节点声明 / 默认值 / 设备可用核数确定核数；设备端用 `cpu_affinity` 把任务绑定到指定核组。同时**把 BLAS/OMP 线程数对齐到分配的核数**，使 numpy/torch 等多线程数值计算真正随核数加速。
- **内存预算**：每任务内存上限 = **实时提交内存 + 任务预算**；预算可由节点声明，或开启 `OPENSAI_AUTO_MEM` 时按任务历史画像自动定档。设备端跨平台施加硬上限（Windows 作业对象 Job Object / POSIX `RLIMIT_AS`），上限恒不低于当前占用，安全可靠。
- **传输带宽**：为每条跨设备传输分配速率（节点声明，或按 `OPENSAI_TARGET_XFER_S` 目标时延结合前驱实测产出自动估算），并按源设备出口预算收口；分配即在源设备记账承诺、完成即归还。
- **任务历史画像**：协调器按算法名对每任务的峰值内存、边际内存、耗时做 EWMA 聚合，用于内存自动定档与记账精度。

---

6. 设备执行端：进程池 + 资源手环（跨平台）

设备把计算交给一个**预建的 worker 进程池**（`ProcessPoolExecutor`，大小由 `OPENSAI_TASK_WORKERS` 指定）：

- **并发 + 复用 + 真并行**：最多 N 个任务并行、worker 循环复用、以多进程吃满多核；池大小（worker_slots）随心跳上报，供放置做容量感知折算。
- **绑核限算力**：每个任务按下发的核组用 cpu_affinity 绑定；池构建时也把各 worker 预绑到不相交的核片。
- **内存限额**：按 §5 的“基线 + 预算”对 worker 进程设硬上限。
- **稳健启动与回退**：进程池在事件循环启动前同步构建并带超时预热自检；若进程池不可用则自动回退主进程内执行，保证设备照常出结果。
- **算法即插件**：算法以 Algorithm_*.py 形式放入算法库，由执行器自动发现加载。现有算法库涵盖计算类（Pi_Calculation、MatMul、ParallelMatMul）、数据搬运类（ProduceBlob、ConsumeBlob、Relay）与感知/推理类（FeatureExtract、Fusion、CNN）等。

  平台亦提供容器化执行后端（resource_control_interface.py，限 CPU/内存、容器起停与复用），可按部署需要启用。

---

7. 设备间数据面（QUIC 直传）

中间结果在设备之间**点对点直传**，不经协调器中转大数据：

- **传输**：发送端 ArtifactServer 注册产物，消费端 fetch_blob 直接拉取（基于 aioquic）；首次使用自动生成自签证书（certs.py）。
- **限速与公平**：发送端按**令牌桶**限制交给 QUIC 的字节速率；**出口公平平分**将设备出口预算（OPENSAI_DATA_RATE_MBPS）在当前活跃流之间动态平分，守住设备出口总量，随流加入/结束实时再平分。
- **链路遥测**：每次真实传输按“字节/耗时”打点得到设备间有效带宽与时延（EWMA，被动）；并可周期性主动探测（OPENSAI_PROBE_INTERVAL）。这些遥测即 §4 放置成本模型的链路输入。

---

8. 实时监测与聚合视图

放置与资源分配的所有输入均来自真实监测：

- **算力/负载**：心跳上报 cores_total / cores_committed、mem_committed_mb、worker_slots、在建任务数、服务器 ↔ 设备 RTT。
- **链路**：设备间有效带宽/时延（§7 遥测）。
- **聚合视图**：协调器 aggregate_monitor_view 把设备心跳与协调器自有账本（放置预留、已承诺出口带宽、在飞任务、算法历史画像）合并为一份权威快照，周期性发布到 Redis 键 OPENSAI_MONITOR_VIEW；scripts/show_monitor.py 可实时查看设备表、链路与各项账本。

---

9. 任务模型与协同感知能力

- **DAG 任务模型**：应用以节点（算法 + 参数 + 前驱/后继）描述为 DAG；子任务状态机 UN_WORK → READY → WORK → FINISH（另有 EARLY_EXIT / DROPPED）驱动整图推进。
- **协同感知端到端闭环**：平台已实现“一张图按区域分到多台设备分别提取特征（真实 torch 模型）→ 特征经 QUIC 数据面汇聚到融合节点拼接分类”的跨设备协同推理。scripts/demo_collab.py 提供端到端演示，scripts/train_collab.py 提供配套模型训练。

10. 跨平台与工程质量

- **跨平台执行**：进程池、绑核、内存硬上限在 Windows 与 POSIX 上均以各自原生机制实现。
- **容错与守限**：协调器按 TTL 回收异常设备占用的放置/带宽预留（`OPENSAI_RESERVATION_TTL`），防账本耗尽；回退执行路径下仍严格守住核数/内存限额。
- **集成回归**：`scripts/test_suite.py` 一键拉起集群并验证 **5 个端到端场景**——基础 DAG 调度、跨设备数据面传输、多源扇入汇聚、就近放置、CIFAR 协同感知，全部通过。

11. 实测效果（相对随机放置基线）

场景	智能放置	完成时间（makespan）变化
网络重（大数据传输，7 节点 DAG）	plan	**-84%**（接近“全聚合”理论上限）
网络重（45 节点错综 lattice ×10 设备）	plan	**-31%**，跨设备传输次数 49.5 → 16
计算重（7 节点 DAG）	plan	**-32%**
异构设备（强/弱算力混合）	容量感知放置	**-27%**
放置 + 资源双维协同	plan + 资源档位	**-34%**（两维收益叠加）
设备出口带宽公平	出口公平平分	出口预算被严格守住（4MB/s 下双流各 ≈2、合计 ≈4）

复现脚本：`scripts/exp_placement_quant.py`、`exp_placement_scale.py`、`exp_collab_adaptation.py`、`exp_compute_adaptation.py`、`exp_dual_compound.py`、`exp_egress_fairshare.py`。

12. 主要可配置项

平台行为通过环境变量灵活配置，便于按部署与负载调优：

维度	变量
放置	`OPENSAI_PLACEMENT`、`OPENSAI_W_XFER`、`OPENSAI_W_LOAD`、`OPENSAI_DEFAULT_BW_MBPS`、`OPENSAI_PROBE_INTERVAL`
算力/内存	`OPENSAI_TASK_WORKERS`、`OPENSAI_DEFAULT_CORES`、`OPENSAI_AUTO_MEM`
带宽	`OPENSAI_DEVICE_BW_MBPS`、`OPENSAI_DATA_RATE_MBPS`、`OPENSAI_TARGET_XFER_S`
容错	`OPENSAI_RESERVATION_TTL`